

AI 모의 면접 서비스 — AI(FastAPI) 파트 완전 해부

코드 흐름 · API 계약 · BE 연동 가이드 | 대상 저장소: [S15P11D202/AI/](#) | 작성일: 2026-07-22

이 문서를 만든 목적

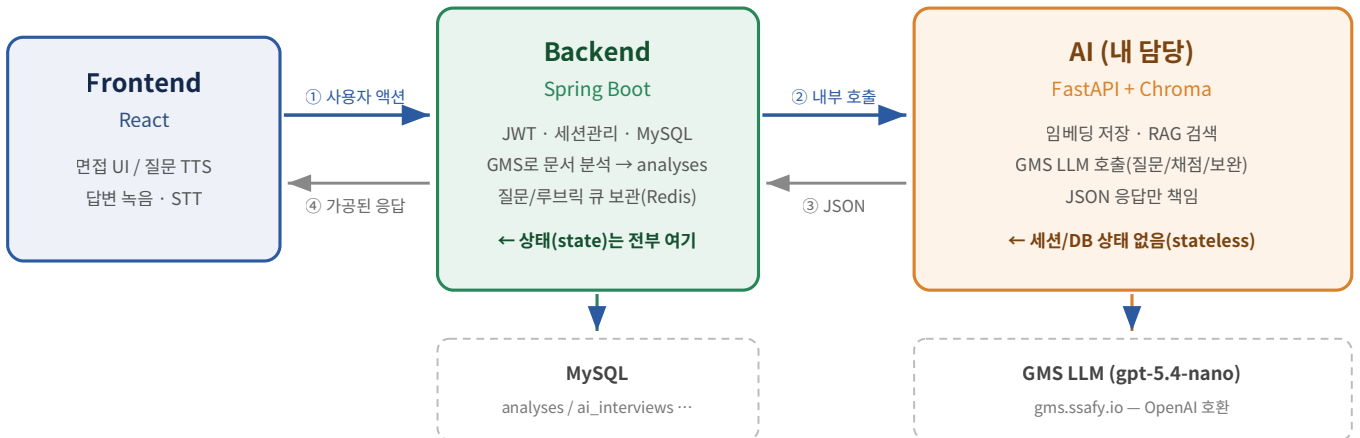
“내가 클로드 코드와 함께 만든 AI 파트 코드를, 처음 요청이 들어오는 순간부터 면접이 끝날 때까지 어떤 API → 어떤 파일 → 어떤 함수 → 어떤 파라미터 순서로 흘러가는지 전부 이해하고, 그걸 근거로 BE(Spring Boot)와 API 계약을 정확히 맞추는 것”이 목표다. 그래서 이 문서는 ① 흐름 그림 → ② 단계별 코드 추적 → ③ 명세서 초안과의 매핑 → ④ BE와 합의할 항목 순서로 구성되어 있다.

목차

1. 큰 그림 — 시스템 경계와 4개의 접점
2. 파일 지도 — 어떤 파일이 무슨 책임을 지는가 (계층 구조)
3. 0단계 — 서버가 뜰 때 벌어지는 일 (`main.py` lifespan)
4. 1단계 — 문서 임베딩 `POST /api/v1/embeddings`
5. 2단계 — 질문 + 루브릭 생성 `POST /api/v1/interviews/questions`
6. 3단계 — 답변 채점 + 꼬리질문 `POST /api/v1/interviews/followup`
7. 4단계 — 답변 보완 `POST /api/v1/interviews/answers/supplement`
8. 부가 API — CS 지식 관리 · 헬스체크 · 임베딩 삭제
9. 전체 타임라인 — 면접 1회의 처음부터 끝까지
10. API 명세서 초안(PDF) ↔ 실제 AI API 매핑표
11. BE와 반드시 맞춰야 할 것 / 아직 없는 것
12. 부록 — 설정값 전체, 데이터 구조, 자주 헷갈리는 포인트

1. 큰 그림 — 시스템 경계와 4개의 접점

가장 먼저 붙잡아야 할 사실 하나: **프론트(React)는 AI 서버를 절대 직접 호출하지 않는다**. FE는 언제나 BE하고만 통신하고, BE가 “AI의 판단이 필요한 순간”에만 내부망으로 FastAPI를 호출한다. 그리고 **AI 서버는 상태를 하나도 갖지 않는다 (stateless)** — 세션·진행상황·이전 답변 같은 건 전부 BE가 들고 있다가 매 요청마다 파라미터로 실어 보내야 한다. AI가 유일하게 들고 있는 영속 데이터는 Chroma 벡터 DB(문서 임베딩)뿐이다.



※ FE — x —> AI : 이 화살표는 존재하지 않는다. FE는 AI의 URL도 스키마도 모른다.

그림 1. 3계층 경계와 호출 방향

AI 서버가 BE에게 열어준 접점은 딱 4개(+관리용 3개)

엔드포인트	언제 호출되나	한 줄 요약
POST /api/v1/embeddings	면접 이전 (문서 저장 직후)	이력서/자소서/GitHub 분석 텍스트를 잘라 벡터로 저장. 이게 되어야 RAG가 의미를 갖는다.
POST /api/v1/interviews/questions	면접 시작 시 1회	RAG 검색 → LLM → 질문 5개 + 질문별 루브릭 2~3개 반환
POST /api/v1/interviews/followup	답변 제출마다 (동기)	루브릭 항목별 pass/fail 채점 + 실패 항목 겨냥 꼬리질문 1개 생성
POST /api/v1/interviews/answers/supplement	답변 제출마다 (비동기 권장)	사용자 답변을 보완한 ai_answer 생성 → 결과 화면용
DELETE /api/v1/embeddings/{user_id}	탈퇴/문서 삭제	해당 유저 벡터 전량 삭제 (먹등)
POST /api/v1/cs-knowledge DELETE /api/v1/cs-knowledge	운영자 수동	CS 전역 지식 전량 재구축 (평소엔 자동 시딩이라 호출할 일 없음)
GET /, /health	인프라	헬스체크

기억할 원칙 3가지

1. **AI는 기억하지 않는다.** 루브릭도, 지금 몇 번째 질문인지도, 꼬리질문을 몇 번 했는지도 BE가 보관하고 매번 실어 보낸다.
2. **인증이 없다.** 현재 코드에 API 키 검증 로직이 없다 — docker-compose 내부망 호출 전제. 외부 노출하려면 인증 계층을 새로 엮어야 한다.
3. **“비동기”는 AI 쪽 스펙이 아니다.** 4개 엔드포인트 전부 FastAPI 입장에선 평범한 동기 요청-응답이다. “비동기로 호출하라”는 건 BE가 응답을 안 기다리고 백그라운드에서 호출하라는 뜻.

2. 파일 지도 — 누가 무슨 책임을 지는가

폴더는 `app/` 없는 평평한 구조다. 그래서 `import`도 접두사 없이 `from config import settings` 형태를 쓴다 (Dockerfile이 `AI/` 폴더 내용물을 `/app`에 복사하고 `uvicorn main:app`으로 띄우기 때문. `from app.`을 쓰면 컨테이너에서 `ModuleNotFoundError`가 난다).

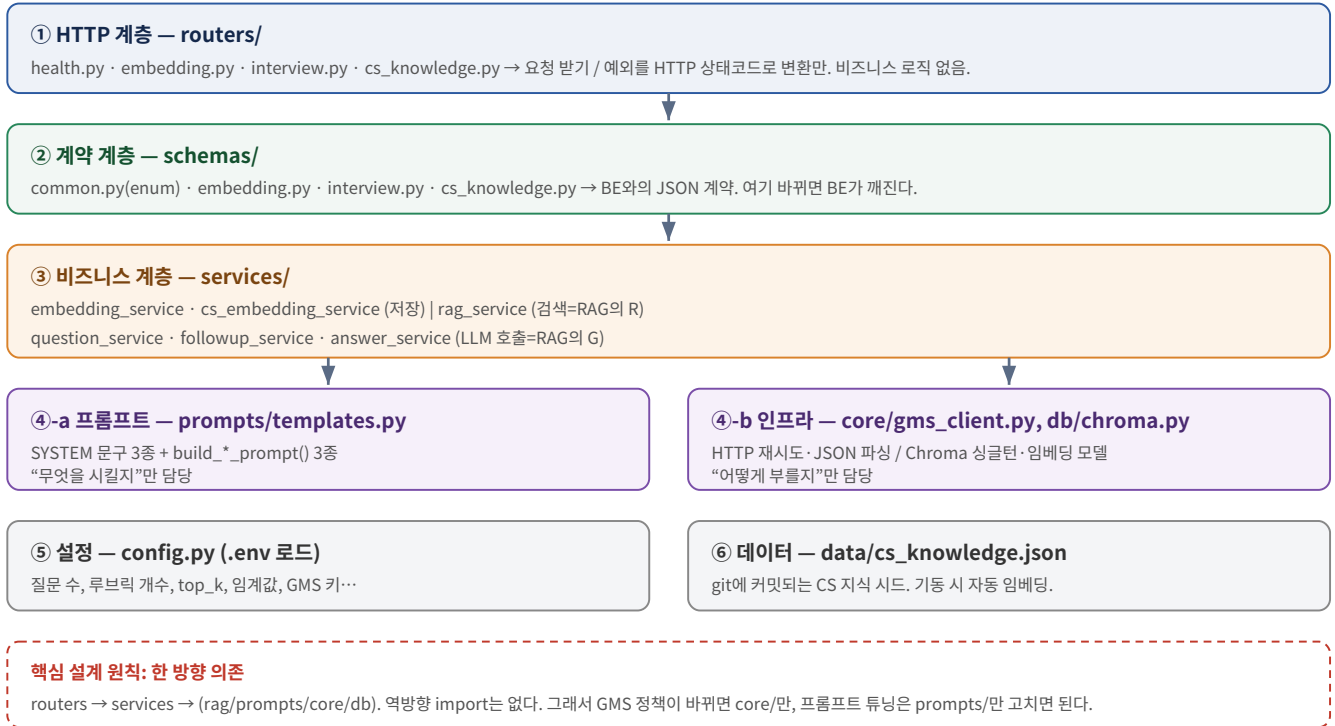


그림 2. 계층별 책임 분리

파일별 한 줄 사전

파일	책임 / 대표 함수
main.py	ASGI 앱 생성, lifespan에서 모델·컬렉션 선로딩 + CS 자동 시딩, 라우터 4개 등록
config.py	<code>Settings</code> (pydantic-settings) — <code>.env</code> 읽어 전역 <code>settings</code> 객체 생성. <code>@lru_cache</code> 로 1 회만
db/chroma.py	벡터DB 유일 통로. <code>get_collection()</code> (개인문서) / <code>get_cs_collection()</code> (CS) / <code>reset_collection()</code> . 클라이언트·임베딩모델·컬렉션 전부 전역 캐싱
core/gms_client.py	<code>chat()</code> / <code>chat_json()</code> . tenacity 3회 재시도, 코드펜스 방어 파서 <code>_safe_json_parse()</code> , 실패는 전부 <code>GMSError</code> 로 통일
schemas/common.py	enum 4종: <code>DocType</code> , <code>InterviewType</code> , <code>Difficulty</code> , <code>CSCategory</code> + <code>CS_CATEGORY_LABEL</code>
schemas/embedding.py	<code>EmbedRequest</code> / <code>EmbedResponse</code> / <code>DeleteEmbeddingResponse</code>
schemas/interview.py	질문·꼬리질문·답변보완 6개 모델 + <code>RubricResult</code> . CS면접 시 <code>cs_category</code> 필수 검증 validator
services/embedding_service.py	<code>_split_text()</code> (청킹) · <code>_make_id()</code> · <code>embed_user_documents()</code> · <code>delete_user_embeddings()</code>
services/rag_service.py	<code>DOC_TYPE_WEIGHTS</code> (유형별 문서 비율) · <code>CS_CATEGORY_RAW_MAP</code> · <code>retrieve_context()</code> · <code>retrieve_cs_knowledge()</code> · <code>retrieve_cs_knowledge_random()</code> · <code>format_context()</code>
services/question_service.py	<code>generate_questions()</code> — CS 분기, 관련도 판정, 루브릭 정규화
services/followup_service.py	<code>generate_followup()</code> — 하드캡 체크 → 재검색 → One-Call 채점 → 서버측 재검증
services/answer_service.py	<code>generate_answer_supplement()</code> — 답변 보완, 실패 시 원문 풀백
services/cs_embedding_service.py	<code>embed_cs_knowledge()</code> · <code>clear_cs_knowledge()</code> · <code>seed_cs_knowledge_if_empty()</code>
prompts/templates.py	<code>INTERVIEW_TYPE_GUIDE</code> · <code>DIFFICULTY_GUIDE</code> + SYSTEM 3종 + <code>build_question_prompt()</code> / <code>build_followup_prompt()</code> / <code>build_answer_supplement_prompt()</code>

3. 0단계 — 서버가 뜰 때 (main.py lifespan)

API 호출이 하나도 없는 시점에 이미 중요한 일이 벌어진다. `uvicorn main:app` 이 앱을 띄우면 `lifespan()` 컨텍스트 매니저의 `yield` 이전 코드가 **딱 한 번** 실행된다.

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    get_collection()      # ① user_documents 컬렉션 + SentenceTransformer 로딩
    get_cs_collection()   # ② cs_knowledge 컬렉션
    seed_cs_knowledge_if_empty() # ③ CS 지식 자동 시딩
    yield                 # ← 여기서부터 트래픽 수신
    logger.info("서버 종료")
```

① / ② 왜 미리 로딩하나

`db/chroma.py` 의 `_get_embedding_function()` 이 `jhgan/ko-sroberta-multitask` 모델을 메모리에 올린다. 이걸 수 초가 걸리는 무거운 작업이다. 만약 첫 API 요청이 들어올 때 lazy 로딩하면 **그 요청을 보낸 BE가 타임아웃**을 맞을 수 있다. 그래서 트래픽 받기 전에 미리 채워둔다. `_client`, `_embedding_fn`, `_collections` 세 개가 모듈 전역 변수로 캐싱되므로 이후 모든 요청은 이 캐시를 재사용한다.

컬렉션 생성 시 메타데이터

`metadata={"hnsw:space": "cosine"}` — 유사도 계산을 코사인으로 고정한다. 그래서 검색 결과의 `distance` 는 **코사인 거리**(= $1 - \text{코사인 유사도}$)이고, **0에 가까울수록 유사하다**. 이 사실이 뒤에 나올 CS 관련도 판정 (≤ 0.45)의 전제다.

③ CS 지식 자동 시딩 — `seed_cs_knowledge_if_empty()`

Chroma 데이터는 Docker 볼륨에 저장되므로 git으로 공유되지 않는다. 그래서 팀원이 클론하면 CS 지식이 비어있다. 이 문제를 **시드 JSON만 git에 커밋하는** 방식으로 푼다.

```
collection = get_cs_collection()
if collection.count() > 0:      # 이미 있으면 스킵 (재시작마다 재임베딩 방지)
    return 0
seed_file = Path(settings.cs_seed_path) # "data/cs_knowledge.json"
if not seed_file.is_file(): return 0
raw = json.loads(seed_file.read_text(encoding="utf-8"))
items = [CSKnowledgeItem(category=..., concept=..., content=...) for r in raw if r.get("content")]
embed_cs_knowledge(CSEndRequest(items=items, replace=False))
```

결과적으로 팀원은 `git pull` → `docker compose up` 만 하면 CS 지식이 자동으로 채워진다. JSON 한 줄의 형식은 `{"category": "...", "concept": "...", "content": "..."}.`

여기까지 정리 — 서버 기동 후 상태: 임베딩 모델 메모리에 로드됨 / `user_documents` 컬렉션 존재(비어있음) / `cs_knowledge` 컬렉션 존재(시드 데이터 채워짐). 이제 BE의 첫 호출을 받을 준비 완료.

4. 1단계 — 문서 임베딩 `POST /api/v1/embeddings`

호출 시점: 사용자가 이력서/자소서를 저장하거나 GitHub를 연동했을 때. BE가 GMS로 분석해서 `analyses` 테이블에 저장한 직후, 그 `content` 를 그대로 시에 넘긴다. 면접보다 훨씬 먼저 일어나는 “사전 준비” 단계다.

요청 / 응답

필드	타입	설명
Request — <code>schemas/embedding.py :: EmbedRequest</code>		
<code>user_id</code>	int (필수)	<code>users.id</code> . Chroma 메타데이터에 박혀서 나중에 검색 필터가 된다
<code>replace</code>	bool (기본 <code>true</code>)	<code>true</code> 면 해당 <code>user_id</code> 의 기존 임베딩을 전부 지우고 새로 넣는다
<code>items[]</code>	list (최소 1개)	임베딩할 문서 목록
└ <code>doc_type</code>	enum	<code>resume</code> <code>cover_letter</code> <code>github</code>
└ <code>target_id</code>	int	원본 PK (<code>analyses.target_id</code> = <code>resumeId</code> / <code>coverLetterId</code> / <code>repold</code>)
└ <code>content</code>	str	실제 임베딩 대상 텍스트 (<code>analyses.content</code>)
└ <code>title</code>	str null	레포명·회사명 등. 프롬프트에 출처로 표시됨
Response — <code>EmbedResponse</code>		
<code>user_id</code> / <code>embedded_count</code> / <code>chunk_count</code> / <code>message</code>	int/int/int/str	<code>embedded_count</code> 는 문서 개수, <code>chunk_count</code> 는 쪼개진 청크 총 개수. 1개 문서가 N개 청크가 되므로 둘은 다르다

```
POST /api/v1/embeddings
{
  "user_id": 7,
  "replace": true,
  "items": [
    { "doc_type": "resume",      "target_id": 12, "title": "백엔드 이력서",
      "content": "Spring Boot 기반 커머스 API 개발...\n\nJPA N+1 문제를 fetch join으로..." },
    { "doc_type": "github",     "target_id": 41, "title": "shop-api",
      "content": "Redis 캐시 도입으로 응답시간 380ms → 90ms..." }
  ]
}
↓
200 { "user_id": 7, "embedded_count": 2, "chunk_count": 9, "message": "임베딩 완료" }
```

코드 추적

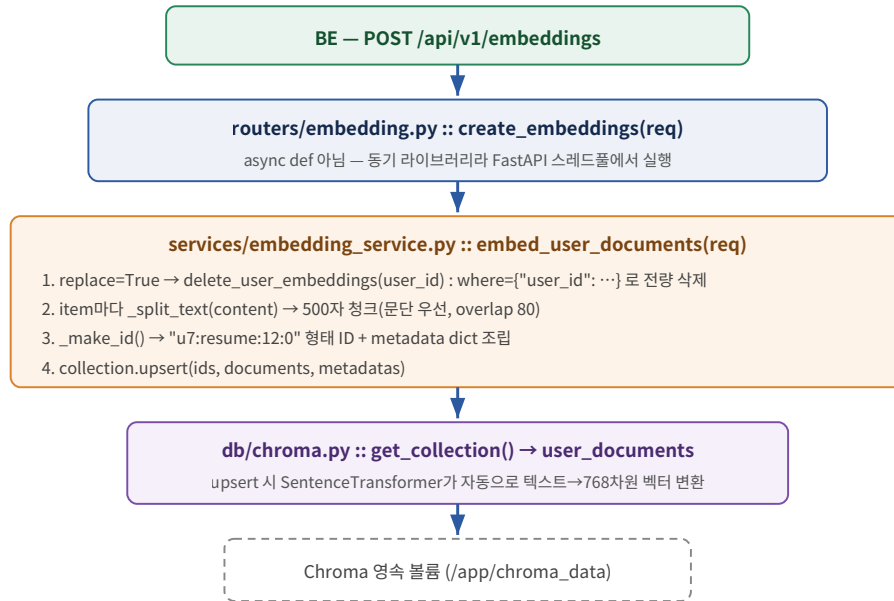


그림 3. 임베딩 저장 흐름

(1) 청킹 — `_split_text()` 가 왜 필요한가

이러서 전체를 벡터 **하나**로 만들면 “프로젝트 A / 프로젝트 B / 자기소개”가 한 덩어리로 뭉개져 검색 정확도가 떨어진다. 그래서 문단(`\n\n`) 경계를 우선 존중하면서 500자 이내로 묶고, 문단 자체가 500자를 넘으면 슬라이딩 윈도우로 강제 분할한다. 이때 **80자 overlap**을 두어 강제 분할 지점에서 문맥이 끊기는 걸 완화한다.

```
CHUNK_SIZE = 500 ; CHUNK_OVERLAP = 80
paragraphs = [p.strip() for p in text.split("\n\n") if p.strip()]
for p in paragraphs:
    if len(buf) + len(p) + 2 <= size: buf = f"{buf}\n\n{p}" if buf else p    # 여러 문단 합치기
    else:
        if buf: chunks.append(buf)
        if len(p) > size:                # 문단 하나가 너무 길면
            while start < len(p):
                chunks.append(p[start:start+size]); start += size - overlap    # ← overlap 반영
            else: buf = p
```

이 함수는 `cs_embedding_service.py` 도 그대로 import해서 재사용한다(청킹 정책 중복 방지).

(2) ID 규칙 — 재임베딩이 자동으로 “갱신”이 되는 이유

```
def _make_id(user_id, item, idx):
    return f"u{user_id}:{item.doc_type.value}:{item.target_id}:{idx}"    # 예: u7:resume:12:3
```

같은 문서(같은 `user_id + doc_type + target_id`)를 다시 임베딩하면 **같은 ID**가 나오고, `upsert` 는 같은 ID를 덮어쓴다. 그래서 “기존 항목 찾아 지우고 넣기” 로직이 따로 필요 없다.

주의 — 청크 개수가 줄어들면 잔여 청크가 남는다

이력서를 수정해서 청크가 9개 → 5개로 줄면, `upsert` 만으로는 예전 `...:5 ~ ...:8` 청크가 그대로 남아 검색에 섞인다. 지금 코드는 `replace=true` 일 때 `user_id` 단위로 전량 삭제해서 이 문제를 피한다. 즉 “한 문서만 갱신하면서 `replace=false`”를 쓰면 위험하다.

BE와 합의할 것: ① 문서 하나를 갱신할 때도 그 유저의 전체 문서를 다시 보내며 `replace=true` 로 호출할 것인지, ② 아니면 AI 쪽에 `target_id` 단위 삭제 로직을 추가할 것인지. 현재 구현은 ①을 전제로 한다.

(3) metadata — 이후 모든 검색 필터의 근거

```
{ "user_id": 7, "doc_type": "resume", "target_id": 12, "title": "백엔드 이력서", "chunk_index": 3 }
```

`user_id · doc_type` 은 `rag_service` 의 `where` 필터로, `title` 은 `format_context()` 의 출처 표시([문서 1 | 이력서 - 백엔드 이력서])로 쓰인다.

(4) 에러 처리

코드	발생 조건
422	Pydantic 검증 실패 (<code>items</code> 비어있음, <code>doc_type</code> 오타 등)
400	모든 <code>item.content</code> 가 공백 → <code>embedded_count == 0</code> . “요청이 잘못됐다”는 뜻으로 500과 구분
500	Chroma 저장 실패 등 서버 내부 예외

5. 2단계 — 질문 + 루브릭 생성 `POST /api/v1/interviews/questions`

호출 시점: BE가 `ai_interviews` 세션 row를 만든 직후, 면접 시작 시 **딱 1회**. 이 한 번의 호출이 이 프로젝트에서 가장 복잡한 로직이다.

요청

필드	타입	설명
<code>user_id</code>	int 필수	RAG 검색 필터 키
<code>ai_interview_id</code>	int 필수	BE가 만든 세션 PK. AI는 저장하지 않고 응답에 그대로 되돌려줄 뿐(로그 추적용)
<code>interview_type</code>	enum 필수	<code>job</code> <code>cs</code> <code>tech</code> <code>portfolio</code> <code>comprehensive</code> — 분기 전체의 기준
<code>difficulty</code>	enum (기본 <code>normal</code>)	<code>easy</code> <code>normal</code> <code>hard</code>
<code>cs_category</code>	enum null	<code>interview_type=cs</code> 면 필수(없으면 422). 9종. 종합 면접에선 선택
<code>company_name</code> , <code>position</code>	str null	지원 기업 / 직무
<code>career</code> , <code>skills[]</code>	str list null	경력·보유 스킬. RAG 쿼리와 프롬프트 양쪽에 반영
<code>question_count</code>	int null	미지정 시 <code>settings.question_count</code> (=5)

응답

```
{
  "ai_interview_id": 101,
  "interview_type": "tech",
  "rag_used": true,
  "questions": [
    { "order": 1,
      "question": "shop-api에서 Redis 캐시를 도입하며 캐시 무효화 전략은 어떻게 설계하셨나요?",
      "rubric": ["캐시 무효화 방식(TTL/이벤트 기반)을 구체적으로 설명했는가",
                "그 방식을 선택한 트레이드오프 근거를 제시했는가"],
      "topic": "Redis", "source": "github" }
  ]
}
```

rubric 이 이 아키텍처의 심장이다. “이 답변이 합격하려면 반드시 짚어야 할 포인트”를 예/아니오로 판정 가능한 한 문장씩, 질문당 2~3개. BE는 이걸 질문과 묶어 **세션 상태(Redis 등)에 저장했다**가 3단계 `/followup` 호출 시 그대로 되돌려줘야 한다. **AI는 이걸 기억하지 않는다**.

`rag_used` 는 “검색 컨텍스트가 실제로 하나라도 있었는가”. `false`면 LLM이 일반 지식만으로 만든 질문이라는 뜻 — 사용자 문서 임베딩이 안 났을 가능성을 의심해야 한다.

코드 추적 — generate_questions()

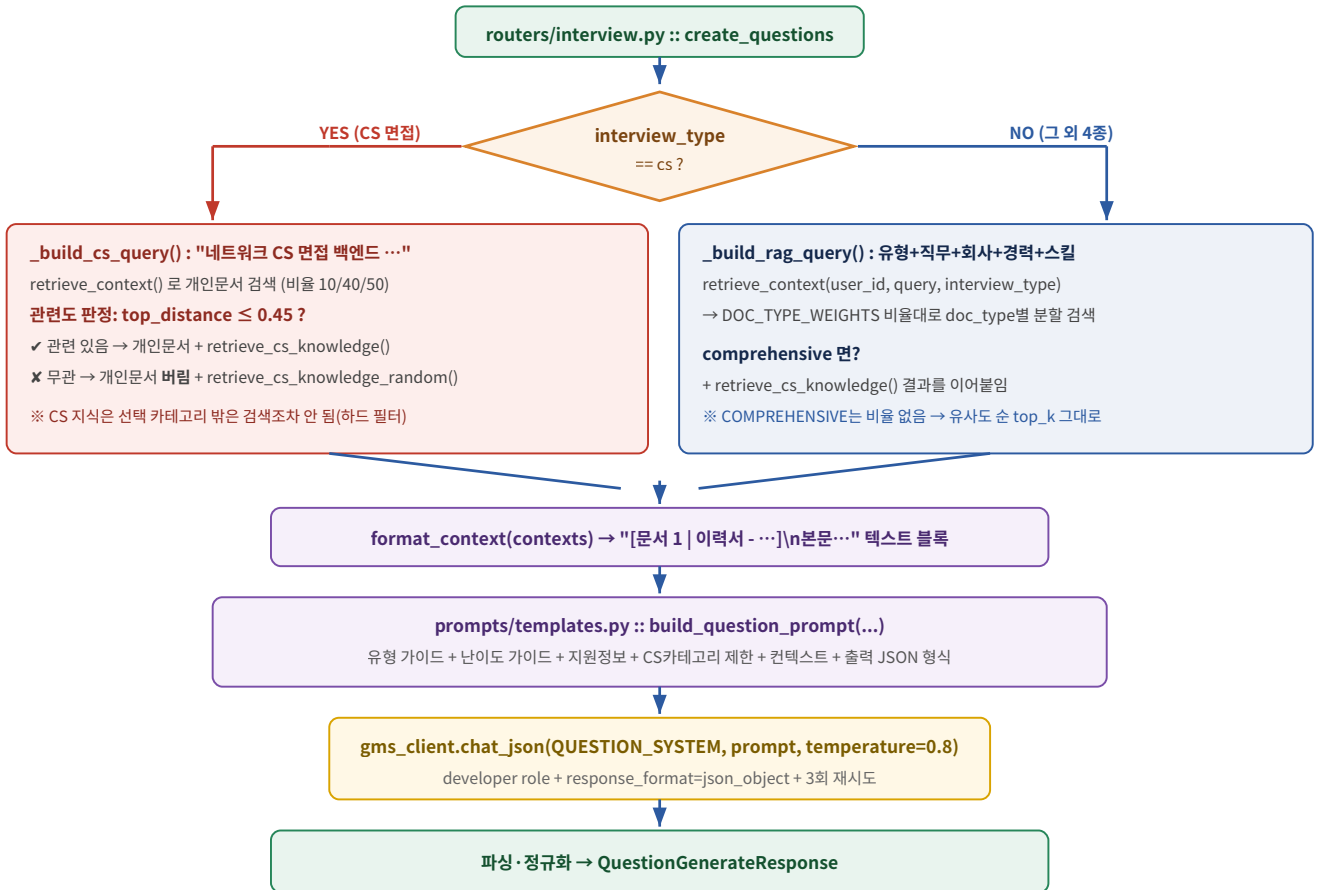


그림 4. 질문 생성 내부 분기

(1) 면접 유형별 문서 참고 비율 — DOC_TYPE_WEIGHTS

면접 유형	이력서	자기소개서	GitHub	top_k=5일 때 실제 배분
job 직무	50%	40%	10%	3 / 2 / 0
cs CS	10%	40%	50%	0 / 2 / 3
tech 기술	10%	40%	50%	0 / 2 / 3
portfolio 포폴	10%	30%	60%	1 / 1 / 3
comprehensive 종합	비율 없음 {} — 유사도 순으로만			5 (구분 없이)

왜 doc_type별로 따로 검색하나? 그냥 where={"user_id":7} 로 top 5를 뽑으면 Chroma가 유사도 순으로만 채운다. GitHub 문서가 우연히 더 비슷하면 이력서 비중이 50%가 아니라 0%가 될 수 있다. 그래서 _allocate_top_k() 로 개수를 먼저 쪼갠 뒤, doc_type마다 별도 쿼리를 날려 합친다.

```

# 최대 나머지방(largest remainder method)
raw = {dt: total_top_k * pct / 100 for dt, pct in weights.items()} # {resume:2.5, cover:2.0, github:0.5}
allocation = {dt: int(v) for dt, v in raw.items()} # {2, 2, 0} → 합 4, 나머지 1
order = sorted(raw, key=lambda dt: raw[dt] - allocation[dt], reverse=True) # 소수점 큰 순
for dt in order[:remainder]: allocation[dt] += 1 # resume에 +1 → {3, 2, 0}

# 그 다음 doc_type마다 개별 쿼리
where = {"$and": [{"user_id": user_id}, {"doc_type": doc_type.value}]}
    
```

비율이 낮은 doc_type이 0개가 되는 건 정상 동작이다(“비율이 낮다”는 의도의 반영). 특히 `followup_service` 가 `top_k=3` 으로 호출할 때 0이 더 자주 나온다.

(2) CS 면접 전용 로직 — 두 겹의 안전장치

결 1: 카테고리 하드 필터. 사용자가 고른 `cs_category` (9종)를 `CS_CATEGORY_RAW_MAP` 으로 시드 데이터의 원본 category 문자열 목록으로 변환하고, Chroma `where={"category": {"$in": [...]}}` 로 건다. 우선순위가 아니라 하드 필터라 범위 밖 지식은 검색 후보에조차 오르지 않는다.

CSCategory	매핑되는 원본 category
<code>data_structure_algorithm</code>	Algorithm / Algorithm > professional / CS > Data Structure
<code>operating_system</code>	CS > Operating System / CS > Computer Architecture (별도 항목이 없어 임의 편입)
<code>network</code>	CS > Network
<code>web</code>	Web (HTTP/REST 등 웹 일반)
<code>database</code>	CS > Database
<code>security</code>	(비어있음) — 시드 데이터 없음
<code>software_engineering</code>	CS > Software Engineering / Design Pattern
<code>ai</code>	New Technology > AI
<code>language_framework</code>	Web > DevOps / React / Spring / Vue

알아둬야 할 구멍: `security`

시드 데이터에 보안 카테고리가 없어서 `CS_CATEGORY_RAW_MAP[SECURITY] = []` 다. Chroma는 `$in: []` 를 유효하지 않은 필터로 보고 예외를 던지므로, `_has_no_seed_data()` 가 **Chroma 호출 전에** 미리 걸러 빈 리스트를 반환한다. 결과적으로 사용자가 “보안”을 고르면 **CS 지식 검색 결과가 항상 0건**이고, 질문은 전적으로 LLM의 일반 지식으로 만들어진다. 서비스 품질 문제이므로 시드 데이터 보강이 필요하다.

결 2: 개인 문서 관련도 판정. ‘보안’을 골랐는데 지원자 문서엔 프론트엔드 얘기만 있다면, 개인 문서를 근거로 질문을 만드는 게 오히려 이상하다. 그래서:

```
top_distance = personal_contexts[0]["distance"] if personal_contexts else None
is_relevant = top_distance is not None and top_distance <= settings.cs_relevance_distance_threshold # 0.45

if is_relevant:
    contexts = personal_contexts
    cs_contexts = retrieve_cs_knowledge(cs_query, cs_category=req.cs_category) # 유사도 검색
else:
    contexts = [] # 개인 문서 통째로 버림
    cs_contexts = retrieve_cs_knowledge_random(req.cs_category) # 카테고리 안에서 무작위 추출
contexts = contexts + cs_contexts
```

`retrieve_cs_knowledge_random()` 이 `query()` 가 아니라 `get() + random.sample` 을 쓰는 이유: Chroma의 `query()` 는 유사도 순으로만 주기 때문에 매번 같은 상위 문서만 나온다. “카테고리 안이면 뭐든 OK”라는 요구사항엔 다양성이 더 중요해서 전체 후보를 가져와 셔플한다. 그래서 이 결과의 `distance` 는 `None` 이다(거리 개념 자체가 없음).

(3) 프롬프트 조립 — `build_question_prompt()`

완성된 프롬프트의 골격은 이렇다. `developer` role로 `QUESTION_SYSTEM` (“당신은 … 면접관 겸 채점 기준 설계자입니다”)을, `user` role로 아래 본문을 보낸다.

다음 조건으로 면접 질문 5개를 생성하세요.

```
## 면접 유형      ← INTERVIEW_TYPE_GUIDE[type]
## 난이도          ← DIFFICULTY_GUIDE[difficulty]
## 지원 정보      ← 기업/직무/경력/보유 스킬
## CS 카테고리 (반드시 이 범위 내에서만 질문) ← cs_category 있을 때만 삽입
## 지원자 문서 (RAG 검색 결과)                  ← format_context() 결과
## 지시사항
- 문서에 실제 등장하는 프로젝트/기술을 근거로. 없는 내용 지어내지 말 것.
- 각 질문마다 rubric을 2~3개, "예/아니오로 판정 가능할 만큼" 구체적으로.
- expected_answer(모범 답안)는 생성하지 말 것.
## 출력 형식 (JSON only) { "questions": [ {order, question, rubric[], topic, source} ] }
```

(4) 응답 파싱 & 방어 로직

- `raw_questions[:count]` — LLM이 더 많이 만들어도 요청 개수까지만 자른다.
- `question` 이 비어있는 항목은 `continue` 로 버린다.
- rubric 정규화: 리스트가 아니거나 문자열이 아닌 항목 제거 → `[:rubric_max_count]` 로 상한 트리밍. **하한(2개)에 못 미쳐도 그대로 둔다** — BE/FE가 실제 개수를 알 수 있도록.
- `order` 는 파싱 후 1부터 **재부여**한다(LLM이 중복/누락한 경우 대비).
- `expected_answer` 는 아예 파싱하지 않는다. LLM이 옛 습관으로 보내도 무시된다.
- `missing_rubric` 개수를 로그로 남겨, LLM이 지시를 무시하는 상황을 조기에 감지한다.

(5) 에러

코드	조건
422	<code>interview_type=cs</code> 인데 <code>cs_category</code> 누락 (<code>model_validator</code> 가 차단) / enum 오타
502	<code>GMSError</code> — GMS 4xx/5xx, 연결 실패, JSON 파싱 실패. BE는 502를 “LLM 쪽 문제”로 인식해 재시도/폴백 정책을 세워야 한다
500	그 외 내부 예외

6. 3단계 — 채점 + 꼬리질문 `POST /api/v1/interviews/followup`

호출 시점: 사용자가 답변을 제출할 때마다(기본 질문/꼬리질문 구분 없이), BE가 **동기로** 호출. 이 응답이 “다음에 뭘 물어 볼지”를 결정하므로 사용자를 기다리게 하는 경로다.

요청 / 응답

필드	타입	설명
Request — FollowupRequest		
<code>user_id</code> , <code>ai_interview_id</code>	int	RAG 필터 / 로그 추적
<code>parent_question</code>	str 필수	직전에 던진 질문 원문
<code>rubric[]</code>	list 최소 1개	2단계에서 받아 BE가 저장해둔 그 질문의 루브릭. 재꼬리질문이면 아직 통과 못한 항목만 추려서 보내는 것을 전제로 설계됨
<code>user_answer</code>	str 필수	STT로 변환된 사용자 답변
<code>interview_type</code>	enum 필수	RAG 비율 + 프롬프트 유형 가이드에 사용
<code>followup_depth</code>	int (기본 0, ≥ 0)	이 질문에 이미 나간 꼬리질문 횟수. 안전장치 전용 — 주 종료조건 아님
Response — FollowupResponse		
<code>rubric_results[]</code>	list	<code>{criterion, passed, reason}</code> . <code>capped=true</code> 면 빈 배열
<code>all_passed</code>	bool	true → BE는 다음 기본 질문으로 , false → 꼬리질문을 큐 맨 앞에 끼워넣기
<code>followup_question</code>	str null	<code>all_passed=true</code> 면 항상 null
<code>followup_depth</code>	int	이번 응답 반영 후 누적값. BE는 이 값을 그대로 저장했다가 다음 호출에 실어 보낸다
<code>capped</code>	bool	상한 도달로 채점 없이 강제 종료했는지. true면 <code>all_passed=true</code> 지만 “진짜 통과”가 아니다

코드 추적 — `generate_followup()` 5단계

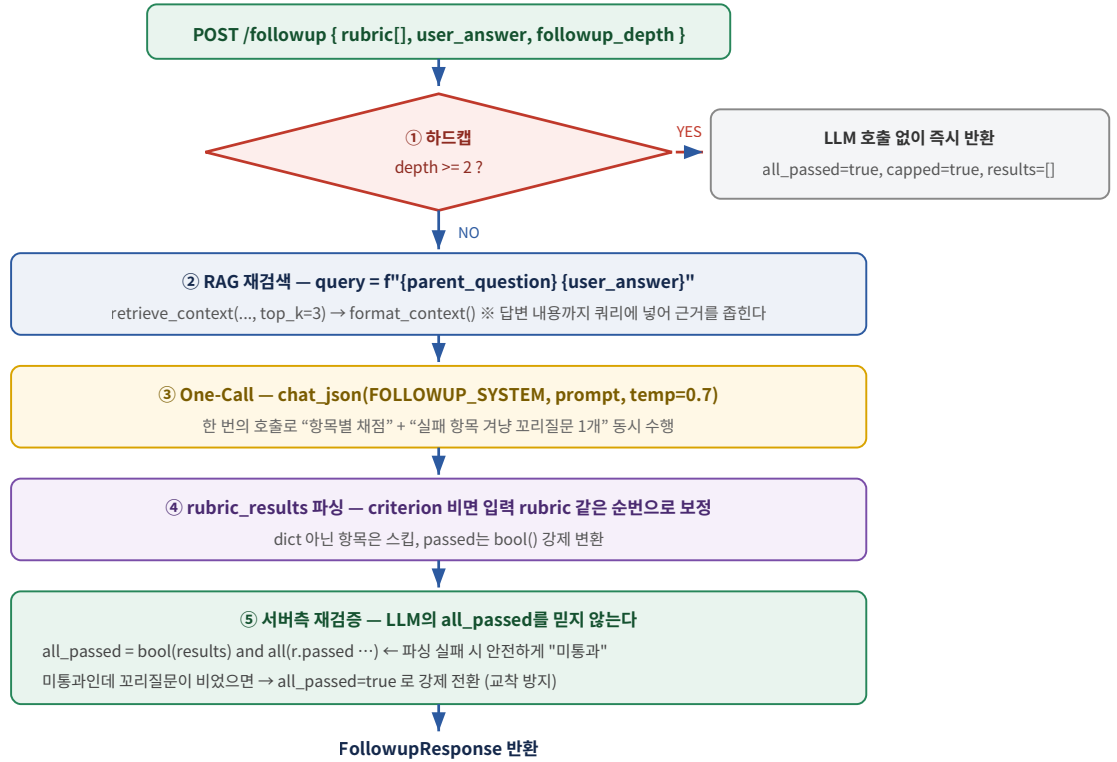


그림 5. 꼬리질문 채점 파이프라인

왜 “LLM이 준 `all_passed`”를 안 믿는가

```

# 5. all_passed 재계산
all_passed = bool(rubric_results) and all(r.passed for r in rubric_results)

# 방어: 미통과인데 꼬리질문이 비어있으면(LLM 누락) → all_passed=True 로 강제
has_followup = bool(followup_question and str(followup_question).strip())
if not all_passed and not has_followup:
    all_passed = True

new_depth = req.followup_depth + 1 if (not all_passed and has_followup) else req.followup_depth
  
```

- 파싱된 결과로 재계산: LLM이 `all_passed:true` 라고 해놓고 개별 항목엔 `passed:false` 를 섞는 모순을 방지.
- `rubric_results` 가 하나도 파싱 안 되면 미통과 취급: 형식 오류를 “통과”로 흘려보내지 않기 위함.
- 교착 방지: “미통과인데 꼬리질문도 없음”이면 BE가 다음 단계로 못 간다. 이 경우 통과로 밀어준다.
- `depth` 는 실제로 꼬리질문이 나갔을 때만 +1. 통과했으면 그대로.

`capped=true` 를 반드시 구분해서 다룰 것

`followup_depth >= 2` 면 LLM을 아예 호출하지 않고 즉시 반환한다. 이때 `all_passed=true` 지만 이는 “다 맞혔다”가 아니라 “더 파고들지 않기로 했다”는 뜻이고, `rubric_results` 는 빈 배열이다. 결과 리포트에서 이걸 “전부 통과”로 표시하면 사용자에게 거짓말이 된다.

BE 쪽 상태 관리 계약 (여기가 제일 헷갈리는 부분)

BE가 보관할 것	이유 / 갱신 규칙
질문 5개 + 각 rubric	2단계 응답을 Redis 큐 등에 저장. AI는 기억 안 함
질문별 followup_depth	3단계 응답의 followup_depth 를 그대로 덮어써 저장 → 다음 호출에 실어 보냄
미통과 rubric 항목	재꼬리질문 시 통과한 항목은 빼고 넘기는 게 설계 전제. (통과한 걸 또 채점할 이유가 없음)
진행 큐	all_passed=false → followup_question 을 큐 맨 앞(LPush)에 끼워 다음 차례로. true → 다음 기본 질문 pop

7. 4단계 — 답변 보완 `POST /api/v1/interviews/answers/supplement`

호출 시점: BE가 사용자 답변을 DB에 저장한 직후, **백그라운드에서 비동기로**. 면접 진행은 3단계 결과로 즉시 이어지고, `ai_answer` 는 나중에 채워진다.

설계가 바뀐 지점 — `expected_answer` 폐기

예전엔 질문을 만들 때 모범답안을 미리 써뒀다(`GeneratedQuestion.expected_answer`). 그런데 “사용자가 아직 답하지도 않았는데 답을 미리 만들어두는 것”이 개념적으로 이상해서 필드를 완전히 제거했다. 대신

`ai_interview_questions.ai_answer` 컬럼의 의미를 “사용자가 실제로 낸 답변을 AI가 보완한 결과”로 재정의했다. 그래서 “새로 쓴 모범답안”이 아니라 “지원자 답변의 보완본”이어야 한다.

요청 / 응답

필드	타입	설명
<code>user_id</code> , <code>ai_interview_id</code>	int	—
<code>question</code>	str 필수	답변 대상 질문 원문 (기본/꼬리 구분 없음)
<code>rubric[]</code>	list (기본 [])	있으면 보완 방향의 기준
<code>rubric_results</code>	list null	3단계 채점 결과를 그대로 실어 보내면 품질이 좋아진다 — 채점점 없이 “어떤 기준을 놓쳤는지” 바로 알
<code>user_answer</code> , <code>interview_type</code>	str / enum	—
Response		
<code>ai_interview_id</code> , <code>question</code> , <code>ai_answer</code>	int/str/str	<code>ai_answer</code> 를 <code>ai_interview_questions.ai_answer</code> 에 저장

코드 추적 — `generate_answer_supplement()`

```
1. query = f"{req.question} {req.user_answer}"
   contexts = retrieve_context(user_id, query, interview_type, top_k=3) # 지원자 실제 경험을 근거로
   context_text = format_context(contexts)

2. rubric_results_dicts = [r.model_dump() for r in req.rubric_results] if req.rubric_results else None
   # Pydantic 모델 → dict 변환 (프롬프트 모듈이 dict를 기대)

3. prompt = build_answer_supplement_prompt(...) # 아래 3단 분기
   data = await gms_client.chat_json(ANSWER_SUPPLEMENT_SYSTEM, prompt, temperature=0.6)
   # ↑ 창의성을 낮게: 지어내면 안 되므로

4. ai_answer = str(data.get("ai_answer") or "").strip()
   if not ai_answer: ai_answer = req.user_answer # 폴백 — 빈 문자열을 DB에 넣어 화면 깨지는 것 방지
```

프롬프트의 3단 분기 — 정보가 있는 만큼만 쓴다

입력 상황	프롬프트에 들어가는 섹션
rubric_results 있음	## 채점 결과 (이미 판정됨 - 미충족 항목 위주로 보완하세요) “- 기준: 미충족 (판정 근거)” 형태로 나열 → 가장 정확
rubric 만 있음	## 채점 기준 (참고) — 목록만 제공
둘 다 없음	## 채점 기준 (제공되지 않음 — 일반적인 답변 품질 기준으로 보완하세요.)

보완 지시사항의 핵심 (환각 방지)

- 지원자 답변의 좋은 부분(본인의 표현)은 그대로 유지
- 미충족 기준에 해당하는 내용만 자연스럽게 추가/보강
- 완전히 새 답변을 쓰지 말 것 — “보완본”이어야 함
- 언급하지 않은 구체 경험(수치, 프로젝트명)을 지어내지 말 것. 보강이 필요하면 “일반적으로는 ~하는 방식으로 접근할 수 있습니다” 형태로
- 3~6문장의 자연스러운 하나의 답변

8. 부가 API

DELETE `/api/v1/embeddings/{user_id}`

사용자 탈퇴/문서 삭제 시. `collection.delete(where={"user_id": user_id})` 한 줄. **삭제 대상이 없어도 실패로 취급하지 않는다(먹등)** — 여러 번 호출해도 안전. 응답: `{user_id, deleted: true, message}`.

POST **DELETE** `/api/v1/cs-knowledge`

운영자/BE가 CS 지식 원본(gyoogle 레포 등)을 갱신해 **전량 재구축**할 때만 쓰는 관리용 API. 평상시엔 서버 기동 시 자동 시딩되므로 호출할 일이 없다. 개인 문서 API와 달리 **user_id가 없다** — 전역 데이터라 “누구의 것”이 아니라 컬렉션 단위로만 다룬다.

- POST body: `{ items: [{category, concept, content}], replace: false }`. `replace=true`면 `reset_collection()`으로 컬렉션을 통째로 지우고 재생성한 뒤 삽입.
- 저장 시 본문 앞에 `[{category}] {concept}\n`을 붙인다 → “TCP 3-way handshake”로 검색할 때 매칭률 상승.
- ID는 `cs:{md5(category:concept)[:12]}:{idx}` — concept에 공백/특수문자가 섞여도 안전한 고정 길이로 정규화.
- 응답: `{embedded_count, chunk_count, total_in_collection}`.

GET `/, /health`

`{"status": "ok"}` 만 반환. **DB도 GMS도 건드리지 않는다** — 외부 의존성 때문에 헬스체크 자체가 느려지거나 실패하는 걸 막기 위함.

9. 전체 타임라인 — 면접 1회의 처음부터 끝까지

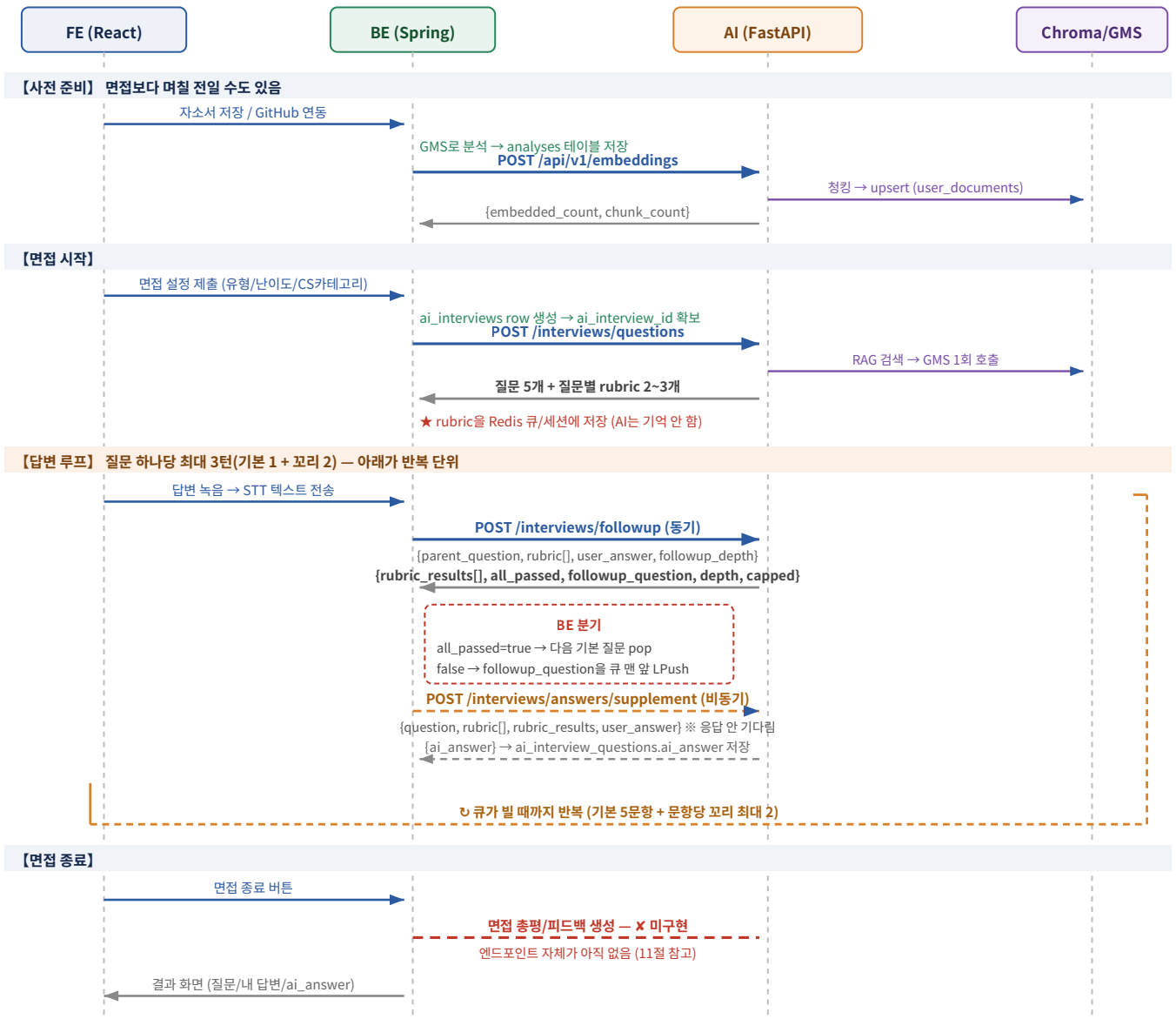


그림 6. 면접 1회 전체 시퀀스 — 실선=동기, 점선=비동기, 붉은 점선=미구현

같은 내용을 “한 문항 처리”로 압축하면

[질문 1 제시]

↓ 사용자 답변

└─ (동기) /followup { rubric: [A, B, C], depth: 0 }

└─ → {A:pass, B:fail, C:pass}, all_passed=false, followup="B를 겨냥한 질문", depth=1

└─ (비동기) /answers/supplement { rubric_results: 위 결과 } → ai_answer 저장

↓

[꼬리질문 제시]

↓ 사용자 답변

└─ (동기) /followup { rubric: [B], depth: 1 } ← ★ 통과한 A, C는 빼고 보냄

└─ → {B:pass}, all_passed=true, followup=null, depth=1

└─ (비동기) /answers/supplement → ai_answer 저장

↓

[질문 2 제시] ...

※ 만약 두 번째도 B를 통과 못하면 depth=2가 되고, 세 번째 답변 시 하드캡에 걸려
LLM 호출 없이 all_passed=true / capped=true 로 강제 종료된다.

10. API 명세서 초안(PDF) ↔ 실제 AI API 매핑

내가 만든 명세서 초안의 “AI 모의 면접” 11개 행은 전부 BE가 FE에게 제공하는 API다. 그중 일부만 내부적으로 AI FastAPI를 호출한다. 아래 표가 그 대응 관계다.

#	명세서 초안 (BE→FE API)	내부적으로 호출하는 AI API	상태 / 메모
1	POST /api/ai-interviews 면접설정 테이블 생성 및 저장	—	BE 단독. ai_interviews row 생성. 여기서 만든 id가 ai_interview_id
2	GET /api/ai-interviews/{id} 저장된 설정 조회 / 최종 확인	—	BE 단독 (MySQL 조회)
3	POST /api/ai-interviews/{id}/questions/generate AI 질문 생성 요청	POST /api/v1/interviews/questions	구현 완료. 응답의 questions[].rubric 을 BE가 반드시 저장해야 함
4	GET /api/ai-interviews/{id}/questions 질문 생성 상태 및 질문 조회	—	BE 단독. 3번에서 받아 저장해둔 걸 조회. “생성 상태”를 두려면 3번을 BE 비동기로 돌린다는 뜻 — 아래 메모 참고
5	POST /api/ai-interviews/{id}/start 모의 면접 시작	—	BE 단독 (status 전환, 큐 초기화)
6	POST /api/ai-interviews/{id}/answers 사용자 답변 저장 및 다음 질문 요청(꼬리질문)	POST /api/v1/interviews/followup (동기) + POST /api/v1/interviews/answers/supplement (비동기)	구현 완료. 명세서엔 1개 행이지만 AI 호출은 2개다. 하나는 “다음 질문”을 정하고, 하나는 ai_answer 를 채운다
7	POST /api/ai-interviews/{id}/behavior-logs 시선·표정·고개 방향 데이터 저장	✕ 없음	AI 미구현. MediaPipe 좌표 기반 MLP / SVM은 MVP 이후. 현재는 BE가 원시 데이터 저장만 하는 것으로 이해
8	POST /api/ai-interviews/{id}/complete 면접 종료 및 AI 분석 요청	✕ 없음	가장 큰 공백. “면접 전체 총평/피드백” 생성 엔드포인트가 AI 쪽에 아직 없다. 11절 참고
9	GET /api/ai-interviews/{id}/analysis-status 분석 진행 상태 조회	—	BE 단독(job 상태). 8번을 비동기 job으로 돌린다는 전제 — “접수됨 + job_id” 계약과 일치
10	GET /api/ai-interviews/results 분석 결과 상세 조회	—	BE 단독 (MySQL 목록 조회)
11	GET /api/ai-interviews/{id}/result 분석 결과 상세 조회	—	BE 단독. 여기서 4단계가 채운 ai_answer 가 노출된다

명세서의 다른 도메인 중 AI와 연결되는 것

명세서 초안	AI API	연결 방식
POST <code>/api/cover-letters</code> · PUT <code>/api/cover-letters/{id}</code> 자기소개서 생성/수정 (완료·박성현)	POST <code>/api/v1/embeddings</code>	BE가 GMS로 분석 → <code>analyses</code> 저장 → 그 직후 AI 임베딩 API 호출. 이 연결이 빠지면 면접 질문이 지원자 문서와 무관한 일반 질문이 된다(<code>rag_used=false</code>).
PUT <code>/api/resumes/{resumeId}</code> 이력서 통합 저장/수정 (시작전·박성현)		
GET <code>/api/github/callback</code> GitHub 연동 (완료·김민범)		
DELETE 회원 탈퇴 / 문서 삭제	DELETE <code>/api/v1/embeddings/{user_id}</code>	명세서에 아직 행이 없다. 탈퇴 시 벡터 정리 호출을 BE 플로우에 추가해야 함
※ 명세서의 커뮤니티/스터디/화상면접/알림 도메인은 AI 파트와 무관하다. 단, 화상면접의 <code>.../ai-summary</code> (상호평가 AI 요약)는 향후 AI 파트로 넘어올 여지가 있으니 담당 범위를 팀과 확인해둘 것.		

명세서 3번 vs 4번에 대한 해석 주의

명세서에 `POST .../questions/generate` (생성 요청)와 `GET .../questions` (생성 상태 및 질문 조회)가 나뉘어 있다. 이 건 “질문 생성을 즉시 응답이 아니라 비동기 job으로 처리한다”는 설계다. AI API 자체는 동기 응답이므로, BE가 POST를 받으면 즉시 `202 + job_id`를 돌려주고 백그라운드 스레드에서 AI를 호출한 뒤 GET으로 폴링하게 만들면 명세서대로 맞는다. 이 부분은 BE와 “동기 응답인가 / job 방식인가”를 명확히 확정해야 한다. (GMS 호출 + RAG 검색이 수 초 걸릴 수 있어 job 방식이 더 안전하다.)

11. BE와 반드시 맞춰야 할 것 / 아직 없는 것

11-1. 지금 당장 BE에 전달해야 할 계약

항목	내용
rubric 보관	질문 생성 응답의 <code>questions[].rubric</code> 을 질문과 묶어 세션 상태에 저장. 이걸 안 하면 /followup 을 호출할 수 없다 (필수 필드, 최소 1개)
미통과 항목만 재전송	재꼬리질문 시 이미 <code>passed=true</code> 인 기준은 빼고 보내는 것이 설계 전제
followup_depth 왕복	응답의 <code>followup_depth</code> 를 저장 → 다음 호출에 그대로 실어 보냄. BE가 자체 계산하지 말 것
capped 구분	<code>capped=true</code> 는 “강제 종료”. 결과 리포트에서 “전부 통과”로 표시하면 안 됨
502의 의미	LLM(GMS) 실패. AI 서버는 이미 3회 재시도한 뒤다. BE는 재시도보다 사용자에게 알리거나 폴백 질문 을 준비하는 게 낫다
cs_category 문자열	9종 enum 값(<code>data_structure_algorithm</code> 등)을 BE가 그대로 보내야 함. ERD에 대응 컬럼이 아직 없는 신규 필드 — 스펙 동기화 필요
replace 정책	문서 갱신 시 해당 유저 전체 문서를 다시 보내며 <code>replace=true</code> 가 현재 전제. 다른 방식을 원하면 AI 쪽 삭제 로직 추가가 필요
인증 부재	AI 서버에 API 키 검증이 없다. 내부망 호출 전제 — 외부 노출 금지

11-2. 아직 구현되지 않은 것 (내 남은 작업)

항목	현황
면접 총평/피드백 리포트	명세서 8번(<code>/complete</code>)에 대응하는 AI 엔드포인트가 없다 . 면접 전체(질문·답변·채점 결과)를 받아 종합 피드백을 만드는 API가 필요. 가장 우선순위 높은 잔여 작업
표정/음성 분석	MediaPipe 좌표 MLP, 음성 떨림 SVM, Whisper 필러워드 — 전부 MVP 이후. 명세서 7번 <code>behavior-logs</code> 가 그 입구
Celery + Redis 파이프라인	모델 학습 이후로 미룸. 다만 API 응답 형태를 “접수됨 + job_id” 로 미리 맞춰두는 것 은 지금 해두는 게 이득
security CS 시드 데이터	매핑이 비어있어 검색 결과 항상 0건. 데이터 보강 필요
GMS 임베딩 API 전환	검토 중. 차원 확인이 선행 조건 — 현 <code>ko-sroberta-multitask</code> 는 768차원이고, Chroma 컬렉션은 첫 벡터의 차원으로 고정되어 이후 못 바꾼다. 차원이 다르면 컬렉션 재생성 + 전체 재임베딩 필요
이력서/자소서 저장 악용 방지	컨텐츠 해시 중복 스킵 / Idempotency-Key / 처리중 락 / Rate Limit 순으로 검토 중. 주로 BE 쪽 책임

12. 부록

12-1. 설정값 전체 (config.py)

키	기본값	영향
gms_model	gpt-5.4-nano	gpt-5 계열이라 지시문을 system 이 아닌 developer role 로 보낸다
gms_base_url	https:// gms.ssafy.io/gmsapi/ api.openai.com/v1	/chat/completions 를 붙여 호출
gms_timeout	60.0초	1회 시도 기준. 재시도 3회면 최악의 경우 훨씬 길어짐
chroma_persist_dir	/app/chroma_data	docker 볼륨. 컨테이너 재시작해도 유지
chroma_collection / chroma_cs_collection	user_documents / cs_knowledge	컬렉션 2개
cs_seed_path	data/ cs_knowledge.json	git 커밋 대상. 자동 시딩 소스
embedding_model	jhgan/ko-sroberta- multitask	768차원. 변경 시 컬렉션 재생성 필요
question_count	5	10 → 5로 축소(루브릭 전환). 요청의 question_count 로 override 가능
max_followup_per_question	2	하드캡. 문항당 최대 3턴
rubric_min_count / rubric_max_count	2 / 3	프롬프트 지시 + 응답 트리밍 상한
rag_top_k	5	개인 문서 검색 개수 (비율대로 분할됨)
cs_top_k	4	CS 지식 검색 개수
cs_relevance_distance_threshold	0.45	이 값보다 코사인 거리가 크면 개인 문서를 버리고 랜덤 CS 풀백. 경험적 값 — 운영하며 튜닝 필요

12-2. GMS 클라이언트가 조용히 해주는 일

- **재시도:** tenacity 로 TransportError / HTTPStatusError 만 3회, 1s→2s→4s 지수 백오프. 4xx도 재시도 대상에 들어가 있는데, 요청 자체가 잘못된 경우엔 무의미하므로 (개선 여지 있음)
- **코드펜스 방어:** response_format=json_object 를 걸어도 LLM이 ```json ... ``` 으로 감싸는 경우가 있다. _safe_json_parse() 가 ① 펜스 제거 → ② json.loads → ③ 실패 시 가장 바깥 {} / [] 블록만 발라내 재시도, 3단계 방어
- **예외 통일:** HTTP 오류·연결 실패·파싱 실패를 전부 GMSError 로 올려보낸다. 그래서 라우터는 except GMSError → 502 한 줄이면 된다

12-3. 자주 헛갈리는 포인트 정리

헛갈리는 것	정답
<code>embedded_count</code> vs <code>chunk_count</code>	앞은 문서 수, 뒤는 청크 수. 문서 1개가 청크 N개(1:N)
Chroma에서 <code>target_id</code> 로 조회하면 1건?	아니다. 청크 개수만큼 여러 벡터가 나온다. <code>collection.get(where={"user_id":..., "target_id":...})</code>
<code>distance</code> 가 클수록 좋은 건가?	반대다. 코사인 거리라 0에 가까울수록 유사
랜덤 CS 검색의 <code>distance</code> 가 <code>None</code> 인 이유	유사도 검색이 아니라 <code>get()</code> + 셔플이라 거리 개념이 없음
<code>all_passed=true</code> 면 다 맞힌 건가?	capped 를 봐야 한다. <code>capped=true</code> 면 채점조차 안 한 강제 종료
라우터가 <code>async def</code> 인 것과 아닌 것	임베딩/CS는 동기(<code>def</code>) — SentenceTransformer가 동기 라이브러리라 FastAPI 스레드 풀로 보내 이벤트 루프 블로킹을 피함. 면접 3종은 async def — GMS 호출이 httpx async
<code>rag_used=false</code> 가 뜨면?	검색 컨텍스트가 0건. 임베딩이 안 됐거나(1단계 누락), CS 카테고리에 시드가 없거나 (<code>security</code>), 관련도 미달로 버려진 경우
<code>from app. import</code>	금지. Docker가 AI/ 내용물을 <code>/app</code> 에 넣으므로 접두사 없이 <code>from config import settings</code>

12-4. 로컬에서 흐름을 눈으로 확인하는 순서

```
# 1) 기동 (로그에 "임베딩 모델 로딩" → "CS 지식 시딩/스킵" → "준비 완료" 확인)
docker compose up --build

# 2) 헬스체크
curl localhost:8000/health

# 3) 문서 임베딩
curl -X POST localhost:8000/api/v1/embeddings -H 'Content-Type: application/json' \
  -d '{"user_id":1,"replace":true,"items":[{"doc_type":"resume","target_id":1,
    "title":"테스트","content":"Spring Boot로 커머스 API를 개발했고 JPA N+1을 fetch join으로 해결했습니다."}]}'

# 4) 질문 + 루브릭 생성 (로그의 rag=True / count=5 / rubric_missing=0 확인)
curl -X POST localhost:8000/api/v1/interviews/questions -H 'Content-Type: application/json' \
  -d '{"user_id":1,"ai_interview_id":1,"interview_type":"tech","difficulty":"normal","position":"백엔드"}'

# 5) 3번에서 받은 rubric을 그대로 복사해 채점 호출
curl -X POST localhost:8000/api/v1/interviews/followup -H 'Content-Type: application/json' \
  -d '{"user_id":1,"ai_interview_id":1,"parent_question":"...", "rubric":["...", "..."],
    "user_answer":"음... 잘 모르겠습니다","interview_type":"tech","followup_depth":0}'
# → all_passed=false 와 followup_question 이 나오는지 확인

# 6) 답변 보완
curl -X POST localhost:8000/api/v1/interviews/answers/supplement -H 'Content-Type: application/json' \
  -d '{"user_id":1,"ai_interview_id":1,"question":"...", "rubric":["..."],
    "user_answer":"음... 잘 모르겠습니다","interview_type":"tech"}'

# 참고: Swagger UI 로 전체 스키마를 눈으로 보는 게 가장 빠르다 → localhost:8000/docs
```

마지막 한 문단 요약

BE가 문서를 분석해 넘겨주면 AI가 **청크로 잘라 Chroma에 저장**(1단계)해두고, 면접이 시작되면 그 벡터를 **면접 유형별 비율대로** 검색해 LLM에게 **질문 5개와 채점 기준을 함께** 만들게 한다(2단계). 사용자가 답할 때마다 그 채점 기준으로 **항목별 pass/fail**을 매기고 **실패한 부분만 겨냥한 꼬리질문**을 한 번의 LLM 호출로 뽑아내며(3단계), 동시에 백그라운드에서 **사용자 답변의 보완본**을 만들어 결과 화면용으로 저장한다(4단계). AI는 이 과정에서 **아무 상태도 기억하지 않고**, 루브릭·진행상황·꼬리질문 횟수는 전부 BE가 들고 다니며 매번 실어 보낸다. 남은 큰 구멍은 **면접 종료 후 총평 리포트 엔드포인트** 하나다.